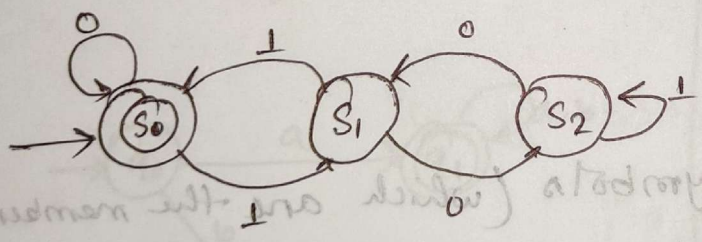
Introduction To Theory of Computing

- It is the study of computational problems that can and cannot be solved using these machines, i.e. what is the extent to which a problem is solvable on a computer.

- Theory of computation can be considered as the study of all kinds of computational model in the field of computer science and it also considers how efficiently the problem can be solved (but not in depth)



Mathematical Defination of Language

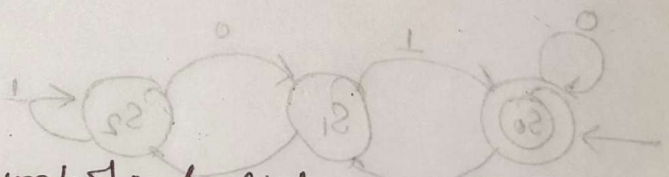
Symbol: Symbols are the basic building block, which can be ~~any~~ any character/token (cow, sheep, ●, ✱, +, ⊗, white flag) (in English we call them letters).

$\{a, b\}$

Alphabet: An Alphabet is a finite non empty set of symbol. In toc, we use symbol Σ for depicting alphabet.

e.g. $\Sigma = \{0, 1\}$. for english

$\Sigma = \{a, b, c, \ldots 2\}$



String: It is sequence of symbols (which are the member of set alphabet).

e.g. $\Sigma = \{a, b\}$ String - aabb, aa, b, so on. (In english we called them as words).

$\Sigma = \{a, b\}$

aa - ON
aaa - OFF
aba - I
baa - II

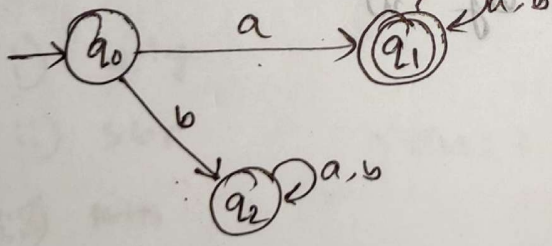
Language: A Language is defined as a set of strings.

$L = \{aa, ab, ba, bb\}$

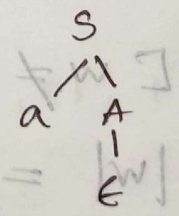
Machine ← Acceptor — Language — generator → Grammar

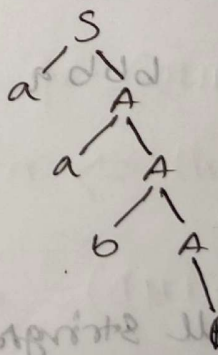
$\Sigma = \{a, b\}$

$L = \{a, ab, aa, aaa\}$



$$\left[\begin{array}{l} S \rightarrow aA \mid a \mid w \mid = |xw| \\ A \rightarrow aA/bA/\epsilon \end{array}\right]$$





Some Basic Operations on Strings

Length of Strings: Denoted like $|w|$.

e.g. length of string $|10010| = 5$

$|aaba| = 4$

$|010| = 3$

concatenation of String: $w = ab$, $x = ba$

$wx = abba$

$xw = baab$ $\quad [w_1 w_2 \neq w_2 w_1]$

$|wx| = |w| + |x|$

Reverse of Strings: w denoted by w^n

$abbb \rightarrow bbba \quad [w \neq w^n]$

$|w| = |w^r|$

Empty / Null Strings: The Strings with zero occurrence of sys' symbols. It is denoted by ϵ, $|\epsilon| = 0$. If there is a string w, then w^n stands for the strings obtained by repeating w to n times.

$A = \{ \} = \emptyset$

$|\epsilon| = 0$

$w^3 = www$

$w^2 = ww$

$w^1 = w$

$w^0 = \epsilon$

$w\epsilon = \epsilon w = w$

Substring: Any string of consecutive symbols in some string 'w' can be collectively said as a substring.

s u b s t r i n g

i) utg

ii) sbr

iii) ~~ntn~~

iv) str

v) sub

vi) ϵ

Consider a string ~~'PEACE'~~ 'FIND' find the total number of substring possible?

→ $\frac{n(n+1)}{2} + 1$

Prefix(FIND) = { ∈, F, FI, FIN, FIND }

$|w| \Rightarrow n+1$

Suffix(FIND) = { ∈, D, ND, IND, FIND }

$|w| = n+1$

if $\Sigma = \{a, b\}$ then find the following ?

$\Sigma^0 = \{\epsilon\}$

$\Sigma^1 = \{a, b\}$

$\Sigma^2 = \{aa, ab, ba, bb\}$

$\Sigma^3 = \{aaa, \ldots\ldots, bbb\}$

- Σ^k is the set of all the strings from the alphabet Σ of length exactly k.

Kleene closure: If Σ is a set of symbols, then we use Σ^* to denote the set of strings obtained by concatenating zero or more symbols from Σ of any length, in general any string of any length which can have only symbols specified in Σ.

$\Sigma^* = \bigcup_{i=0}^{i=\infty} \{w \mid |w| = i\}$ {w using the symbols from the alphabet Σ}

$\Sigma = \{a, b\}$

$\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \ldots\ldots \cup \Sigma^\infty = \Sigma^*$

Positive closure:

$$\Sigma^+ = \bigcup_{i=1}^{i=\infty} \{ w \mid |w| = i \}$$

Q. Given the Language $L = \{ab, aa, baa\}$, which of the following strings are in L^*?

1) abaa baaabaa
2) aaaabaaaa
*3) baaaaabaaaaab
4) baaaaaabaa

A) 1, 2, 3
b) 2, 3, 4
✓ C) 1, 2, 4
D) 1, 3, 4

$\Sigma = \{a, b\}$

$\Sigma^* = \{\epsilon, a, b, aa, ab, \ldots\}$

$L^* = \{\epsilon, ab, aa, baa, abaa, \ldots\}$

Q. The number of substrings that can be formed from the string given by 'adefbghnmp' is

a) 10 b) 45 c) 55 d) 56

Q. Suppose $L_1 = \{10, 1\}$ and $L_2 = \{011, 11\}$ How many elements are there in $L = L_1 L_2$

a) 4 b) 3 c) 2 d) none of these

Chomsky Hierarchy, which consists of four types of grammar

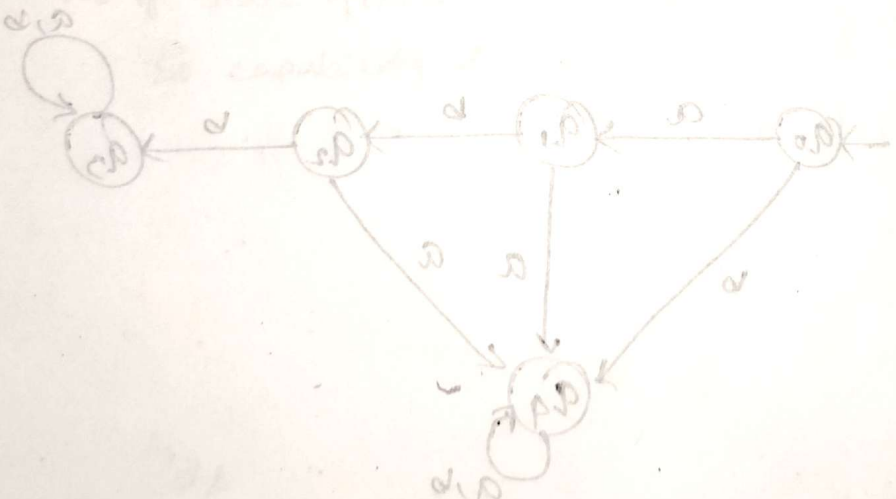
Grammar

(RG) Regular Grammar → Regular Language (RL) → Finite Automata (FA)

(CFG) Context Free Grammar → context free language (CFL) → Pushdown Automata (PDA)

(CSG) context-sensitive grammar → Context Sensitive Language (CSL) → Linear Bounded Automata (LBA)

(REG) Recursively Enumerable Language Grammar → Recursively Enumerable Language (REL) → Turing Machine (TM)

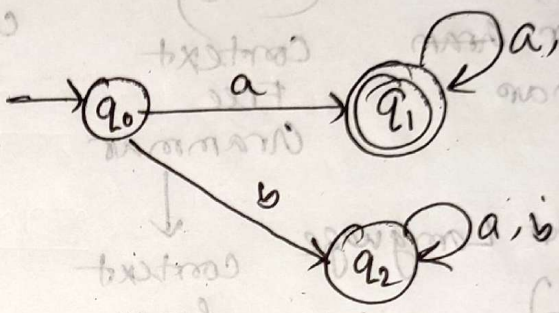


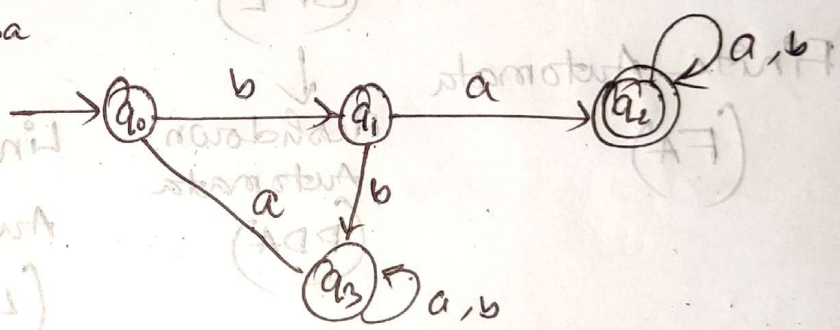
* Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must start with a substring w

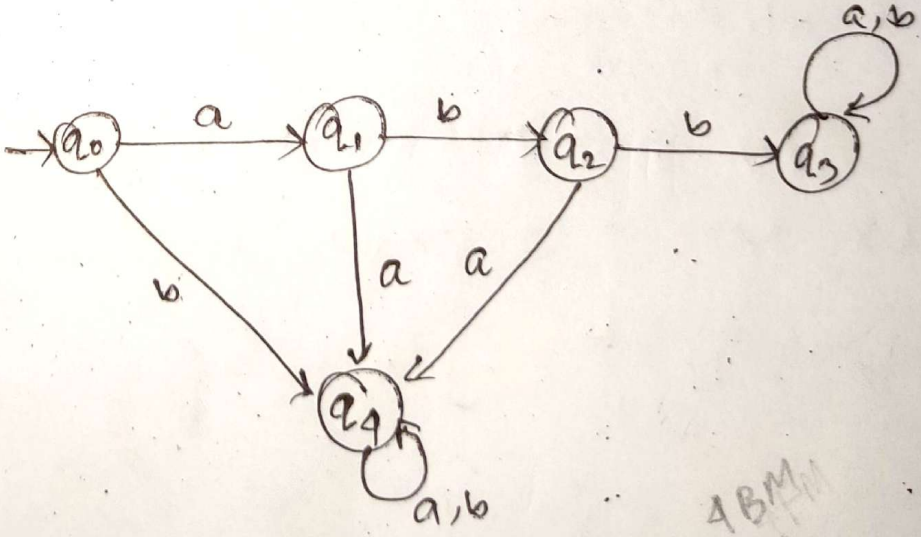
i) a

ii) ba

iii) abb

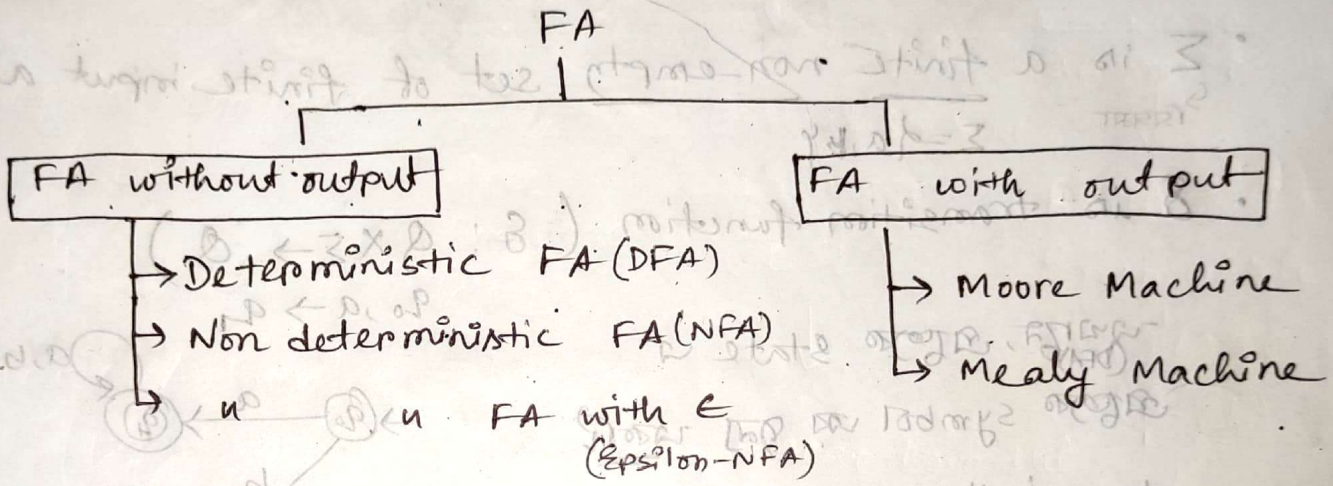






FINITE AUTOMATA (FA)

A finite Automaton is a model that has a finite set of states (represented in the figure by circle) and its control moves from one state to another state in response to external inputs (represented by arrows).



→ e.g: মুই, এক্কারেপক্কার

→ No auxiliary memory

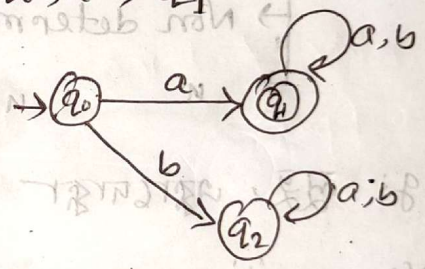
→ No of state finite → memory finite
So capability finite

Deterministic Finite Automata (DFA)

A deterministic FA (DFA) is defined by 5-tuple $(Q, \Sigma, \delta, s, F)$ where

- Q is finite and non-empty set of states
- Σ is a finite non-empty set of finite input alphabet
  (বর্ণমালা) $\Sigma = \{a, b\}$
- δ is transition function $(\delta : Q \times \Sigma \to Q)$
  $q_0, a \to q_1$

  DFA তে, প্রত্যেক state এ
  প্রত্যেক symbol এর জন্য একটি
  transition হবে

  NFA → more than one or no transition.

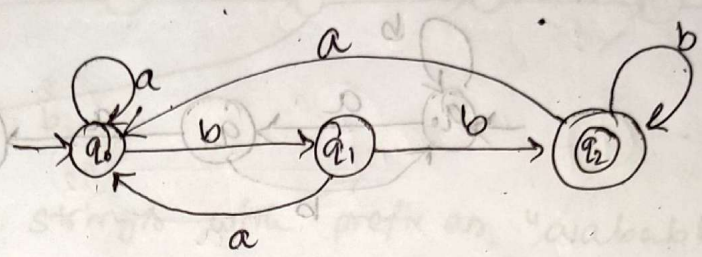
- s is initial state (always one) $s \in Q$
- F is a set of final states $(F \subseteq Q)$ $(0 <= |F| <= N)$

Q. Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must ends with a substring w

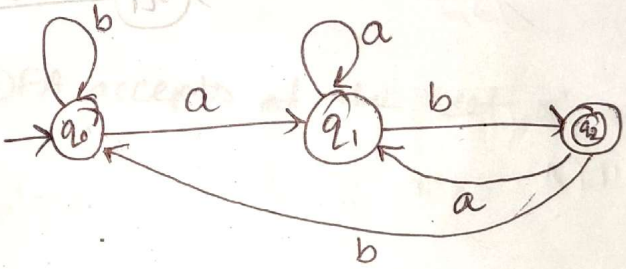
(i) w = bb $L = \{bb, abb, abbb, bbb, babb, \ldots\}$

~~(ii) w = ab~~

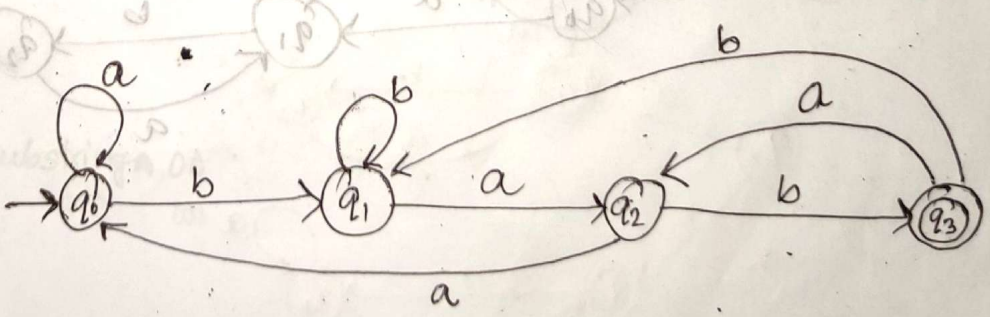
~~(iii) w = bab~~



(ii) w = ab $L = \{ab, aab, bab, \ldots\}$

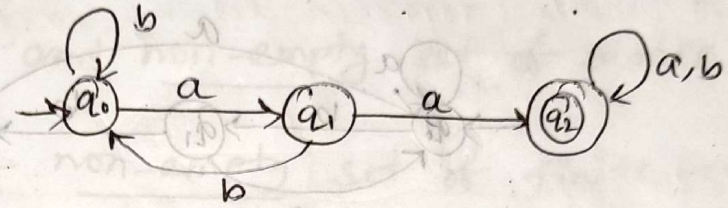


(iii) w = bab $L = \{bab, abab, bbab, \ldots\}$



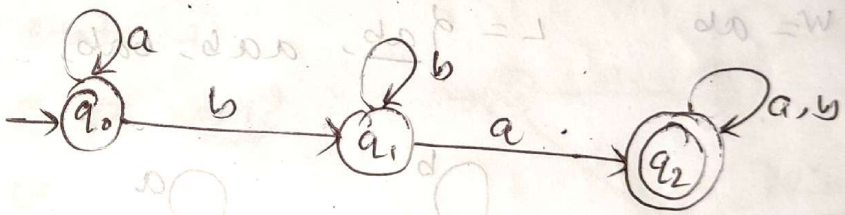
Q. Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must contains a substring w.

(i) w = aa $L = \{aa, aaa, baa, aab, baab, \dots\}$

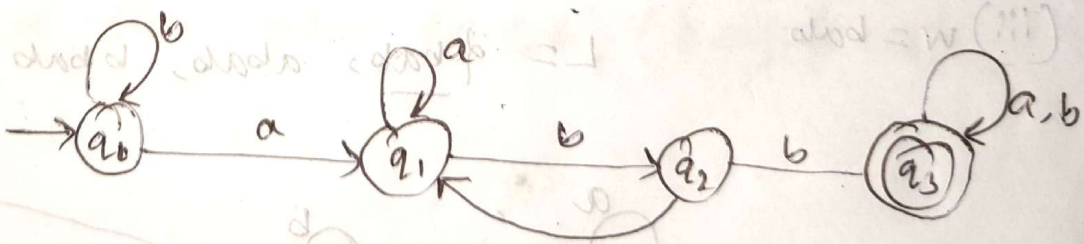


(ii) w = ba

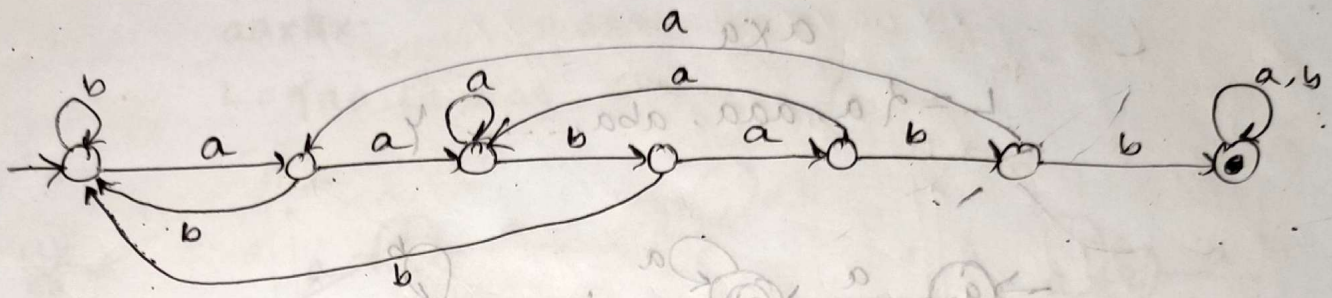
$L = \{ba, aba, bba, baa, bab, \dots\}$



(iii) w = abb



Q. Consider the following Deterministic Finite Automata. Which of the following is true?



XA) It accepts all strings with prefix as "aababb".

✓B) It accepts all strings with substring as "aababb".

XC) It accepts all string with suffix as "aababb".

D) None of the Above.

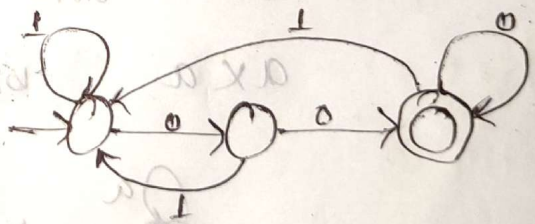
Q. The following DFA accepts at the set of all strings over {0,1} that

a) Begin either with 0 or 1

b) End with 0

c) End with 00

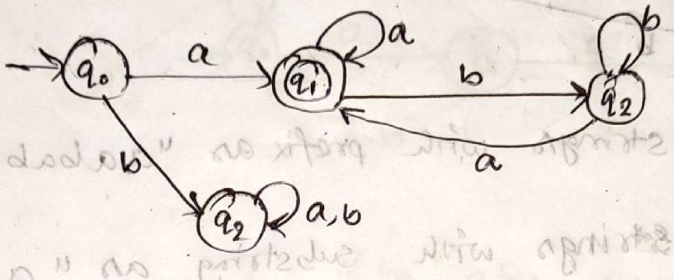
d) Contain the substring 00



Q. Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must start and ends with 'a'.

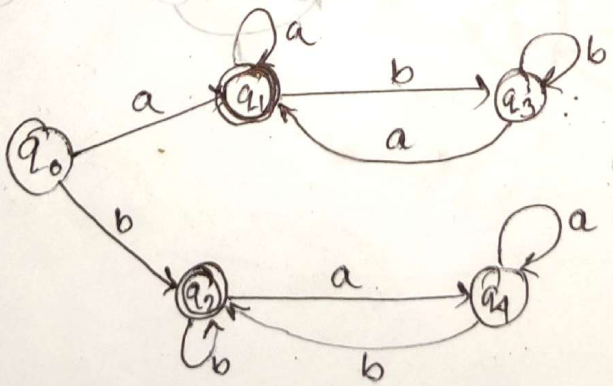
$a \times a$

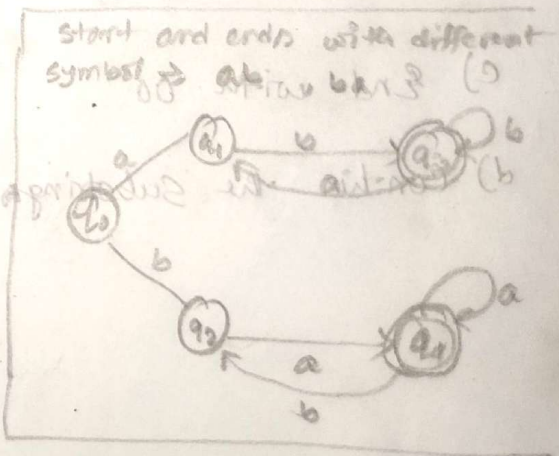
$L = \{a, aaa, aba, \ldots\}$



Q. Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must start and end with same symbol.

$a \times a$, $b \times b$, a, b

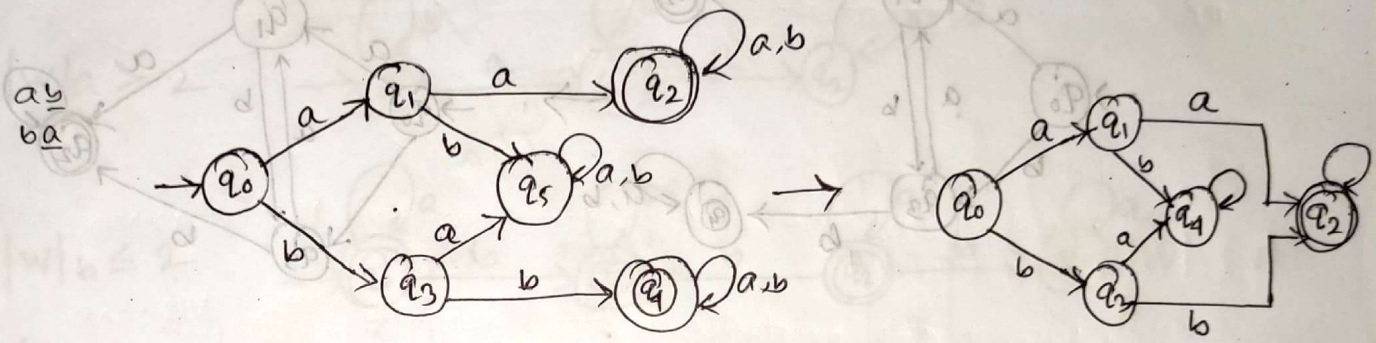




* Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must start with aa or bb

aaxxx... bbxx

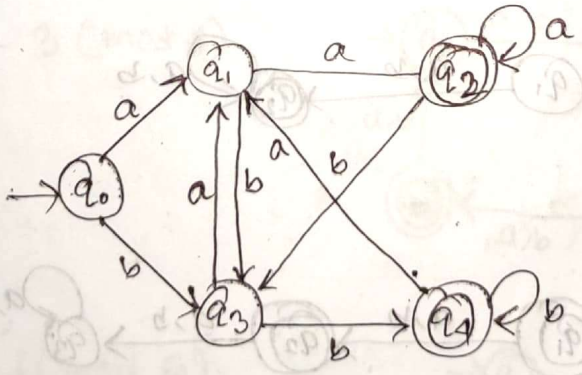
$L = \{aa, bb, aab, bba, \ldots\}$



* Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted ends with aa or bb.

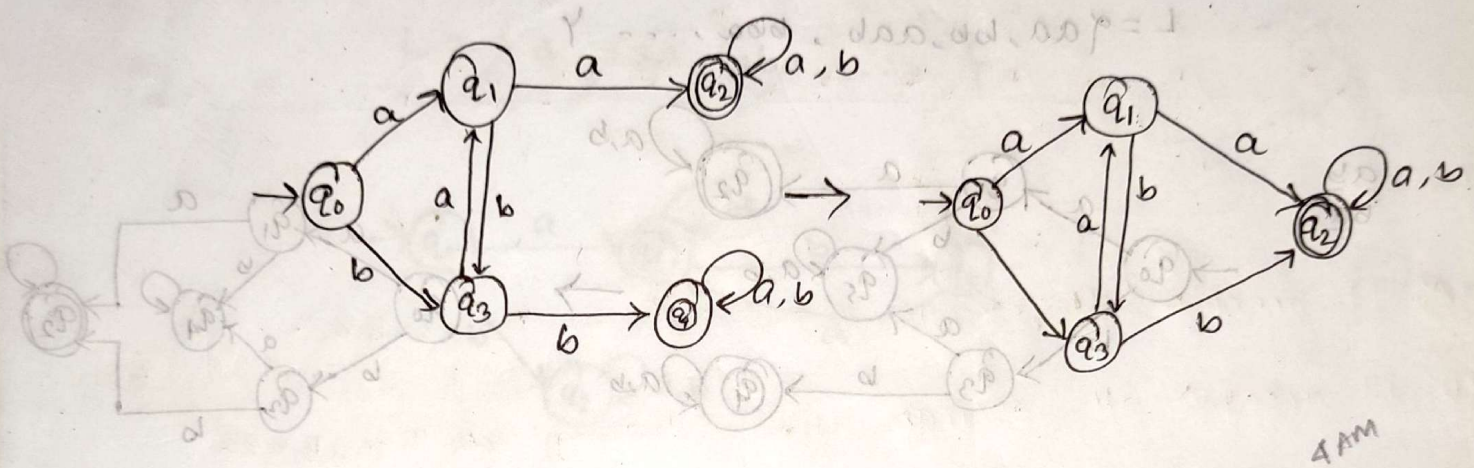
xxaa, xxbb

$\{aa, bb, baa, abb, \ldots\}$



• Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must contains a substring aa or bb

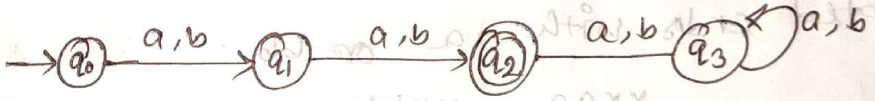
$xx\,aa\,xx$ $\qquad$ $xx\,bb\,xx$



4AM

• Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must

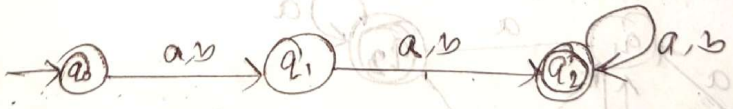
(a) $|w| = 2$



no. of states

$|w| + 2 = 4$

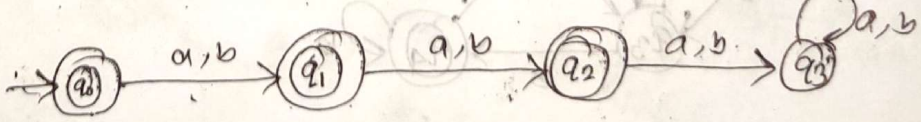
(b) $|w| \geq 2$



no of states

$|w| + 1 = 3$

(c) $|w| \leq 2$

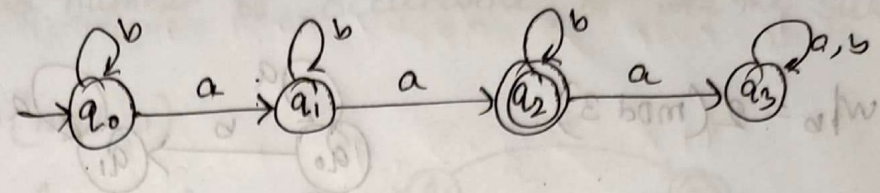


no of states

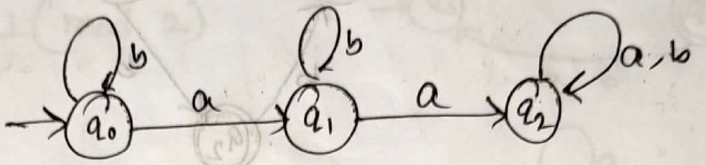
$|w| + 2 = 4$

• Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must

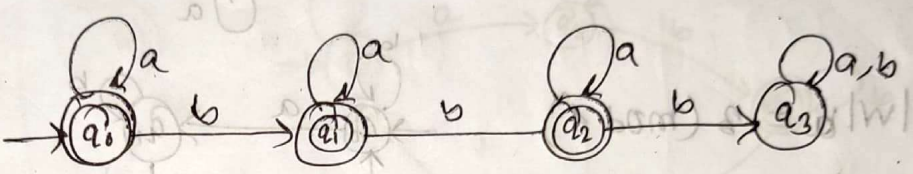
(a) $|w|_a = 2$



(b) $|w|_a \geq 2$



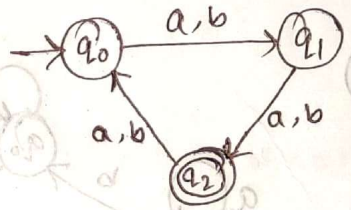
(c) $|w|_b \leq 2$



• Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must

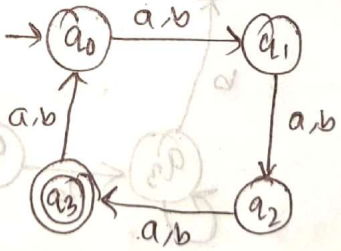
(a) $|w| = 2 \pmod 3$

$|w| = r \pmod n$

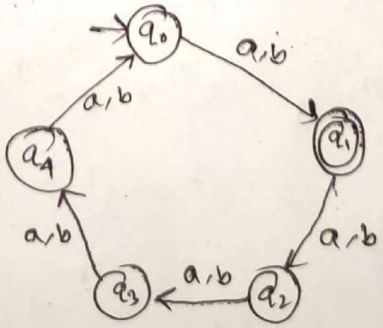


if modulus is 3 then remainder must be 0, 1 or 2

(b) $|w| = 3 \pmod 4$

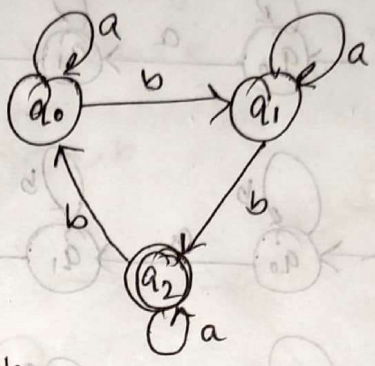


(c) $|w| = 1 \pmod 5$

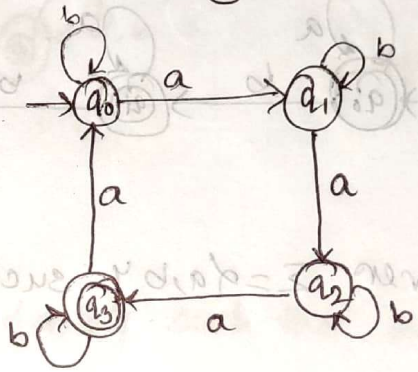


• Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must

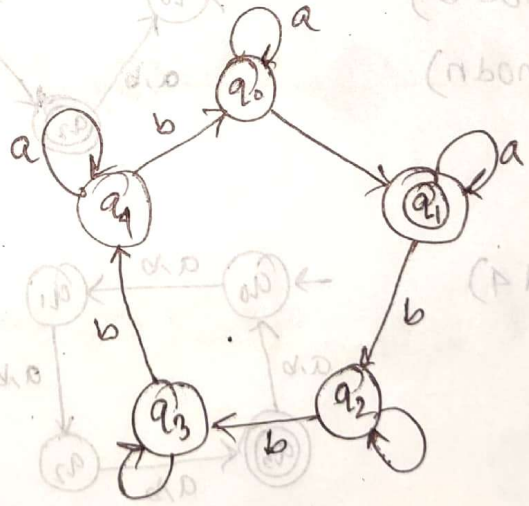
(a) $|w|_b = 2 \pmod 3$



(b) $|w|_a = 3 \pmod 4$



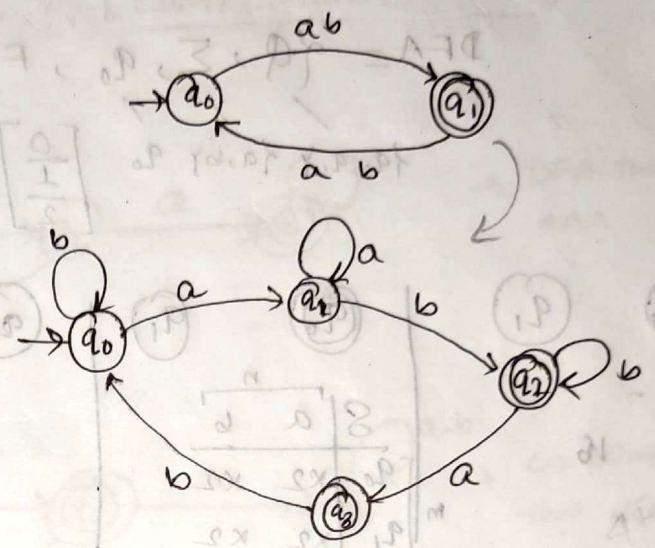
(c) $|w|_b = 1 \pmod 5$



• Design a MDFA over $\Sigma = \{a, b\}$ such that every string accepted must contain odd number of occurance of ~~sth~~ the substring 'ab'

(a) $|w|_x = n \pmod n$

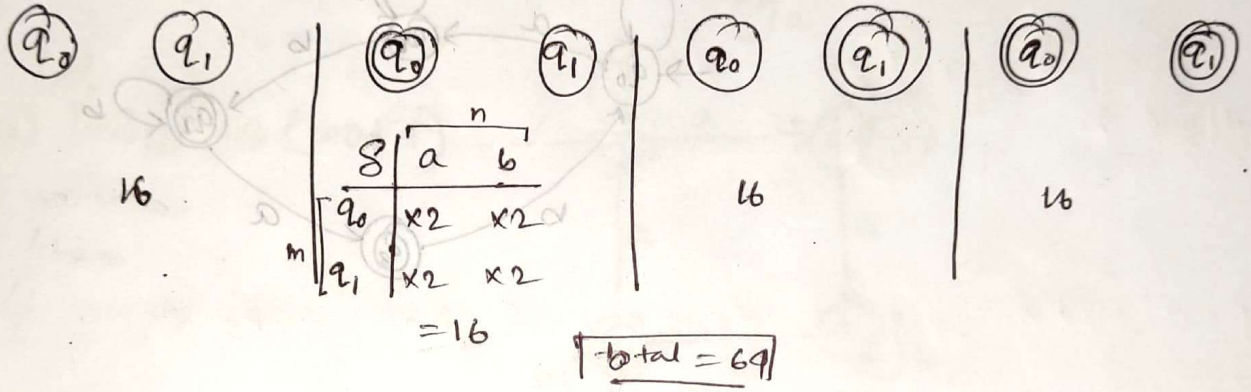
$|w|_{ab} = 1 \pmod 2$



How many different DFA can be designed with fixed initial state over $\Sigma = \{a, b\}$ and number of state is two.

$$DFA = \{Q, \Sigma, q_0, F, \delta\}$$

$\{q_0, q_1\}$ $\{a, b\}$ q_0 $\begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}$ $\delta : Q \times \Sigma \to Q$



total = 64

total states $|Q| = m$ total symbol $|\Sigma| = n$

total state 2 there combination is 4 } 2^m
" 3 will " be 8 }

	a
q_0	×2

m, n

here 2 written because total states are 2 and choice also 2.

$$m^{mxn} = 2^{2 \cdot 2} = 2^4 = 16$$

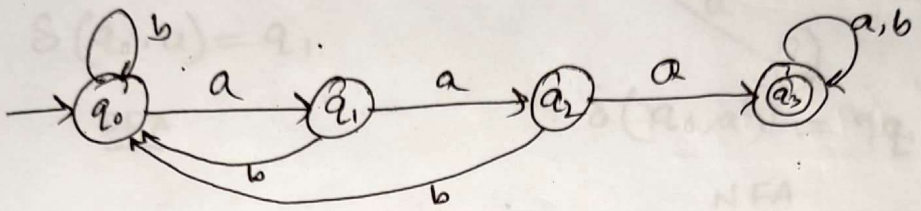
$\therefore$ total combination $= 2^m \times m^{m \cdot n}$

$= 2^2 \times 2^{2 \cdot 2}$

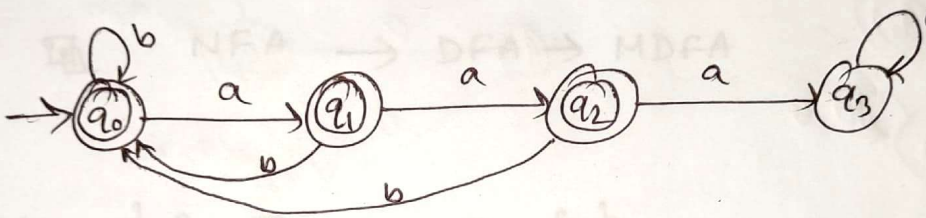
$= 4 \times 16$

$= 64$

• Design a minimal DFA that accepts all the string over the alphaket $\Sigma = \{a, b\}$, such that every string accepted must not contain a sub string aaa ? [COMPLIMENT OF DFA]

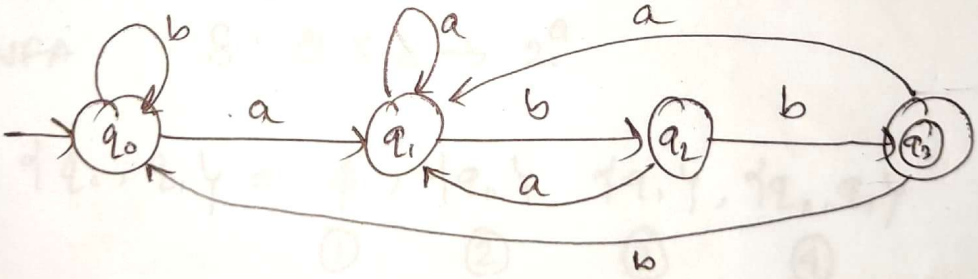


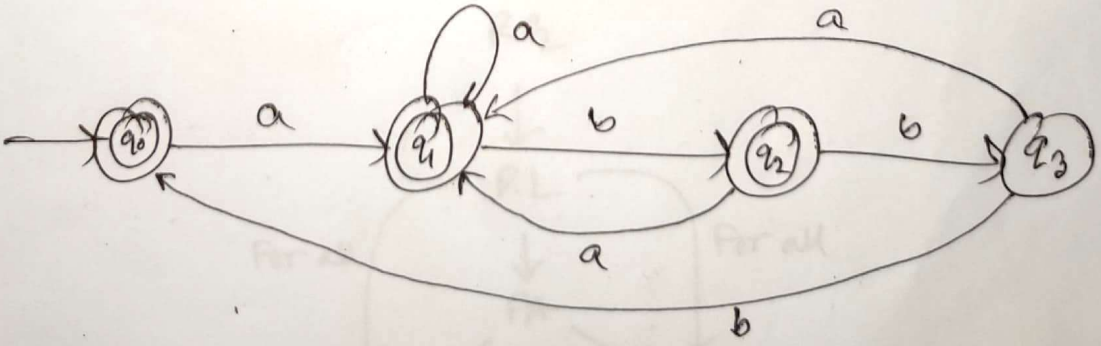
→ DFA that accept aaa



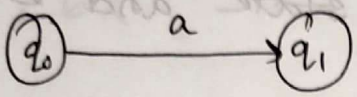
→ compliment of DFA that doesn't accept aaa

abb ends not accept



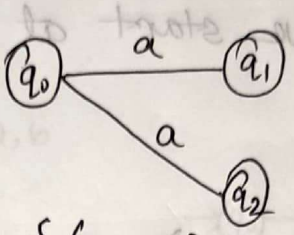


Non-deterministic Finite Automata (NDFA/NFA)



$\delta(q_0, a) = q_1$

DFA



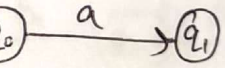
$\delta(q_0, a) = \{q_1, q_2\}$

NFA

NFA → DFA → MDFA

$NFA = \{Q, \Sigma, q_0, F, \delta\}$

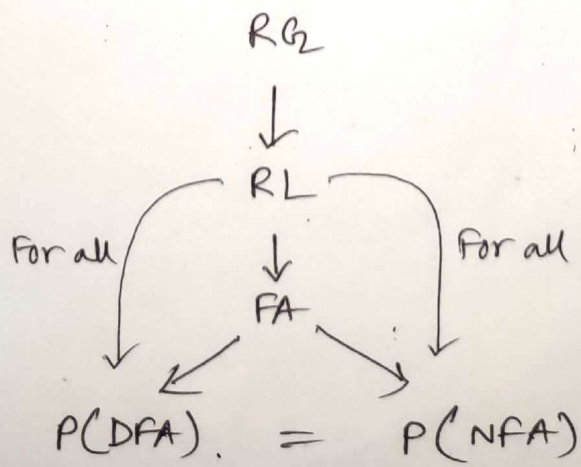
in DFA $\delta : Q \times \Sigma \to Q$



in NFA $\delta : Q \times \Sigma \to 2^Q$

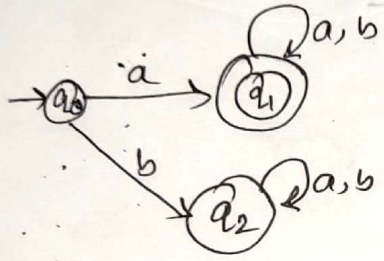
$\{q_0, q_1\} = \phi, \{q_0\}, \{q_1\}, \{q_0, q_1\}$

① ② ③ ④

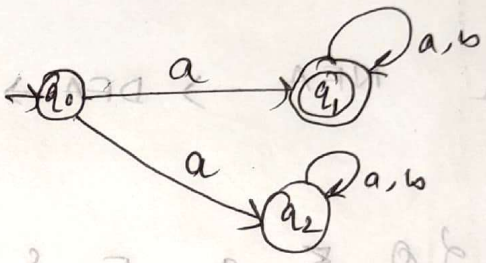


Accepted by NFA: A string 'w' is said to be accepted by a NFA if there exist at least one transition path on which we start at initial state and ends at final state.

$$\delta^*(q_0, w) = F$$



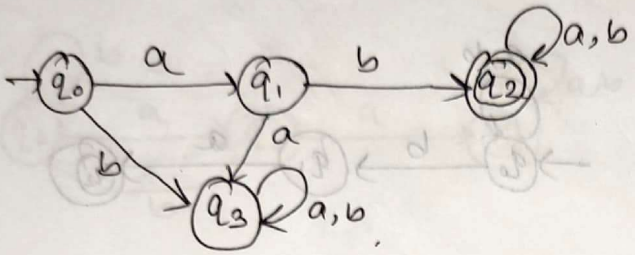
DFA



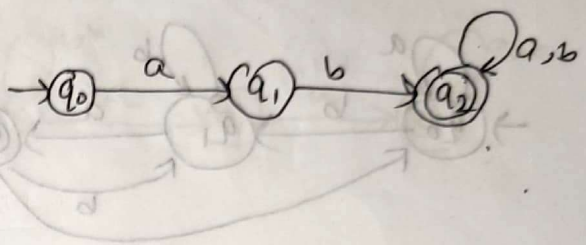
NFA

• Design a NFA over an alphabet $\Sigma = \{a, b\}$ such that every string is accepted must start with

(i) $ab\,x$

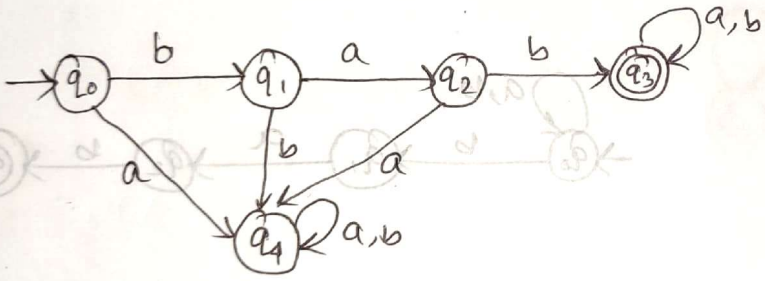


DFA



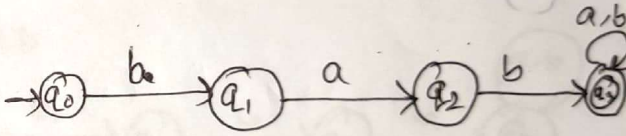
NFA

(ii) $bab\,x$



DFA

$|w| = n+2$

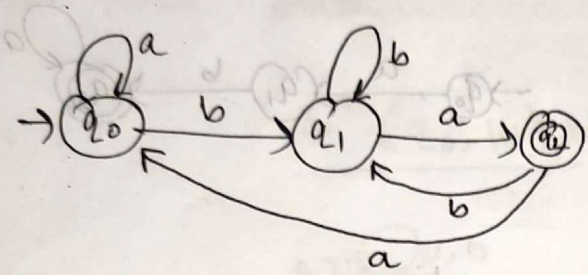


NFA

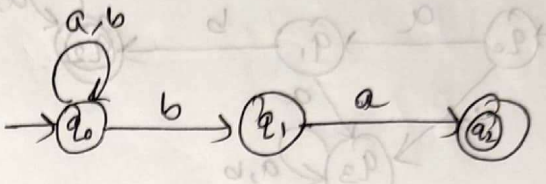
$|w| = n+1$

• Design a NFA over an alphabet $\Sigma = \{a, b\}$ such that every string accepted must ends with

(i) xba

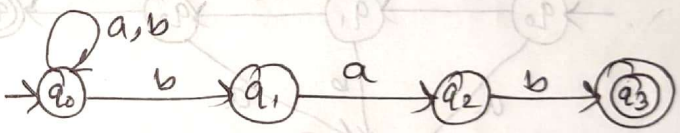


DFA



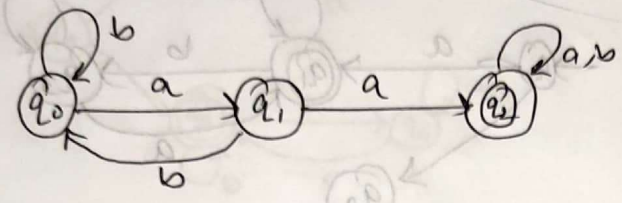
NFA

(ii) xbab

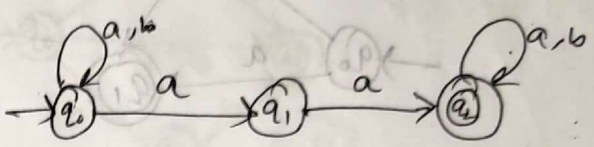


- Design a NFA over an alphabet $\Sigma = \{a, b\}$ such that every string accepted must contain a substring.

(i) xaax

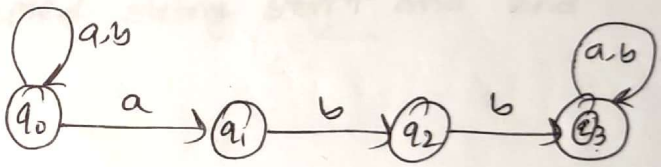


DFA



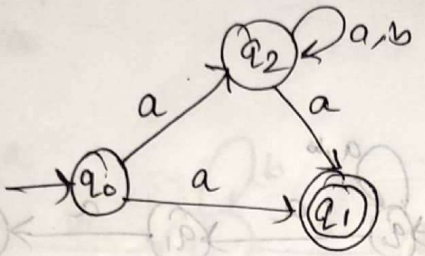
NFA

(ii) xabbx

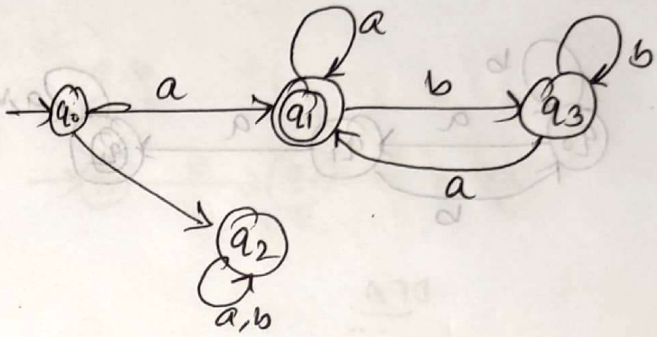


NFA

- Design a NFA over $\Sigma = \{a, b\}$ such that it accepts every string starting and ending with a

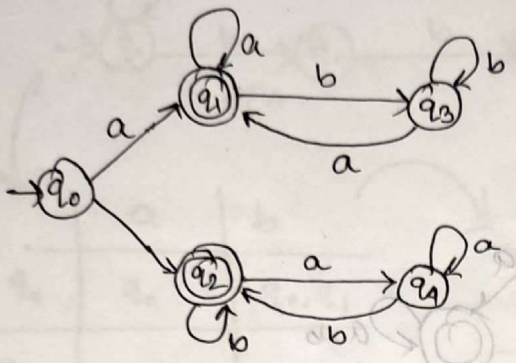


NFA

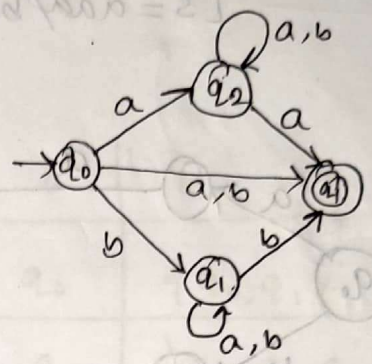


DFA

• Design a NFA over $\Sigma = \{a, b\}$ such that it accepts every string starting and ending with same symbol

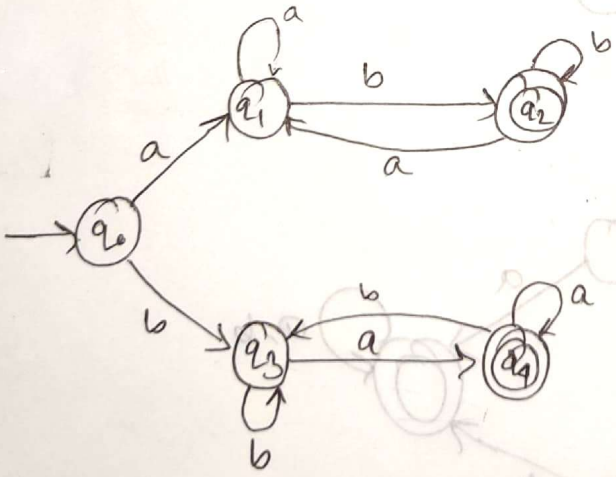


DFA

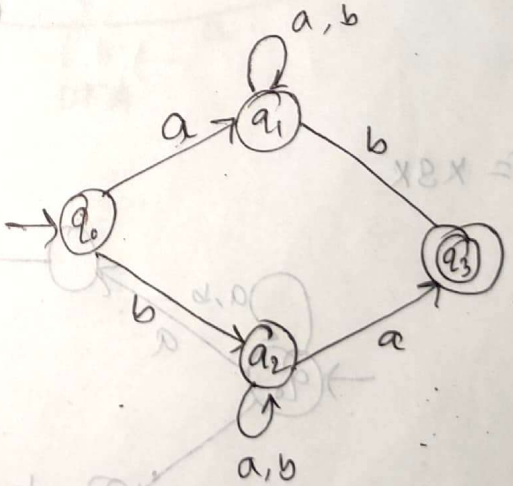


NFA

• Design a NFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with different symbol



MDFA

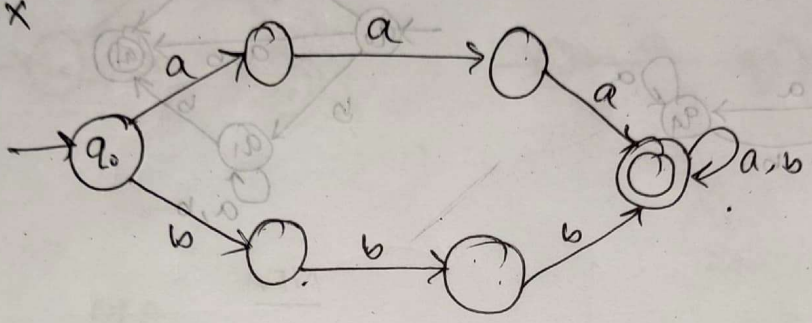


NFA

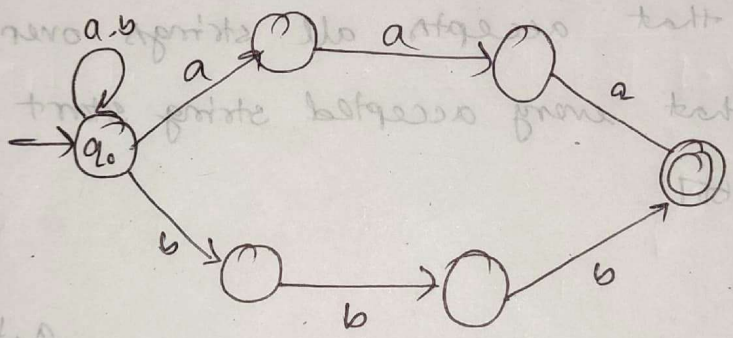
Design a minimal NDFA that accepts all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string w, is like ---

[s = aaa/bbb]

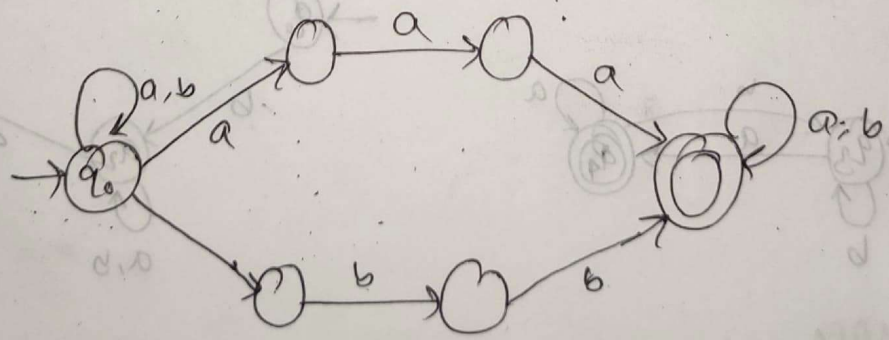
(i) w = sx



(ii) w = xs

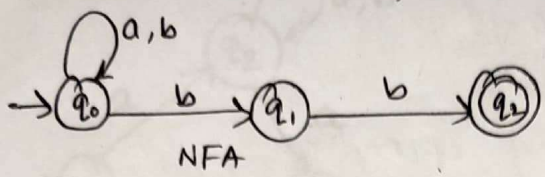


(iii) w = xsx



4EM
4FM

Conversion of NFA to DFA:

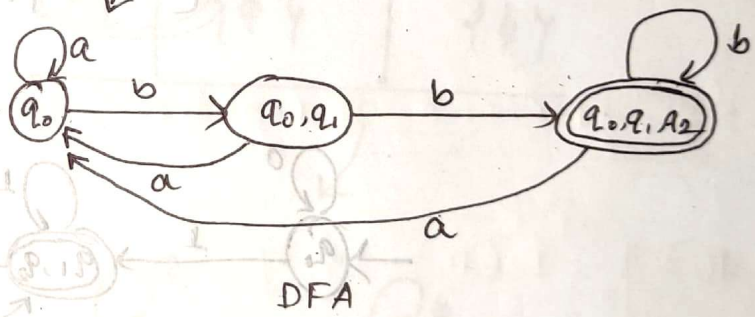


	a	b
→ q_0	q_0	q_0, q_1
q_1	–	q_2
(q_2)	–	–

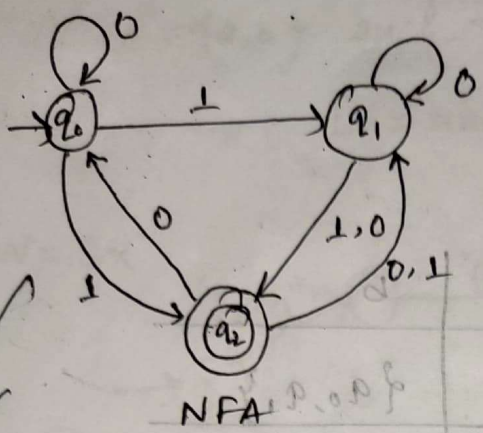
NFA Table (transition table)

δ	a	b
→ q_0	q_0	$\{q_0, q_1\}$
$\{q_0, q_1\}$	q_0	$\{q_0, q_1, q_2\}$
($\{q_0, q_1, q_2\}$)	q_0	$\{q_0, q_1, q_2\}$

DFA Table



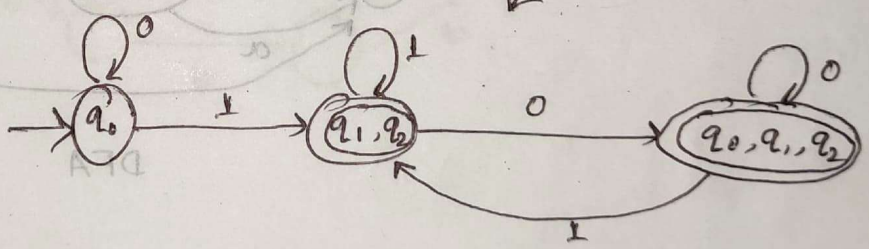
Conversion of NFA to DFA:



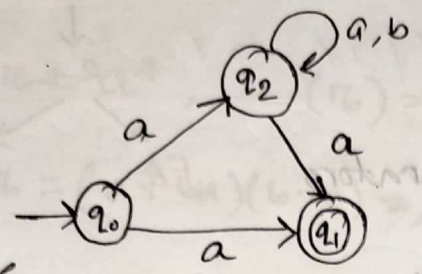
NFA

	0	1
→ q_0	q_0	q_1, q_2
q_1	q_1, q_2	q_2
(*) q_2	q_0, q_1	q_1

	0	1
→ q_0	q_0	$\{q_1, q_2\}$
$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$
(*) $\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$

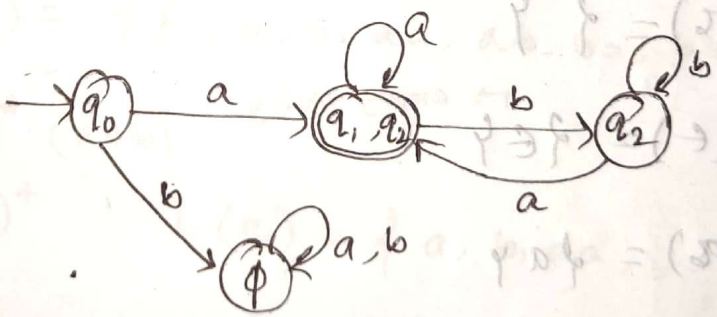


Conversion of NFA to DFA



	a	b
→ q_0	q_1, q_2	–
(q_1)	–	–
q_2	q_1, q_2	q_2

	a	b
→ q_0	$\{q_1, q_2\}$	$\{\phi\}$
(q_1, q_2)	$\{q_1, q_2\}$	q_2
q_2	$\{q_1, q_2\}$	q_2
$\{\phi\}$	$\{\phi\}$	$\{\phi\}$

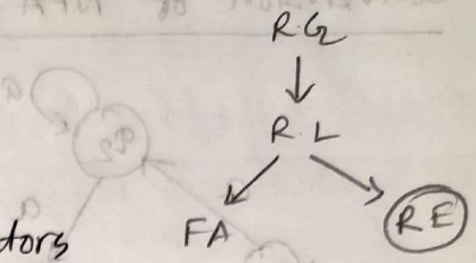


DFA

Regular Expressions:

→ a way of representing R.L

→ Expression of strings and operators



Precedence

(i) * Kleen closure [a*] → a* = ε, a, aa, aaa, ...

(ii) + Positive closure [a+] → a+ = a, aa, aaa ...

(iii) . Concatenation [a.b] → ab.cd = abcd

(iv) + Union [a+b] → a+b = a, b

[a+ε = a, ε
a+ϕ = a] Note

Regular Expression to Regular Language:

(1) r = ϕ, L(r) = {}

(2) r = ε, L(ε) = {ε}

(3) r = a, L(a) = {a}

(4) r = a+b, L(r) = {a, b}

(5) r = a.b, L(r) = {ab}

(6) r = a+b+c, L(r) = {a, b, c}

(7) r = (ab+a).b, L(r) = {abb, ab}

(8) $r_6 = a^+$, $L(r_6) = \{a, aa, aaa, \ldots\}$

(9) $r_6 = a^*$, $L(r_6) = \{\epsilon, a^1, a^2, a^3, \ldots\}$

(10) $r_6 = (a+ba)(b+a)^*$, $L(r_6) = \{ab, aa, bab, baa\}$

(11) $r_6 = (a+\epsilon)(b+\phi)$,

$= (a+\epsilon)(b)$, $L(r_6) = \{ab+b\}$

(12) $r_6 = (a+b)^2$

$= (a+b)(a+b)$, $L(r_6) = \{aa, ab, ba, bb\}$

(13) $r_6 = (a+b)^*$

$= (a+b)^0 + (a+b)^1 + (a+b)^2 + \ldots$

$= \epsilon + \{a, b\} + \{aa, ab, ba, bb\}$

in math $(a+b)^0 = \perp$, but in here it ϵ

$\therefore L(r_6) = \{\epsilon, a, b, aa, ab, ba, bb, \ldots\}$

(14) $r_6 = (a+b)^* \cdot (a+b)$

$= (a+b)^+$ $L(r_6) = \{a, b, aa, \ldots\}$

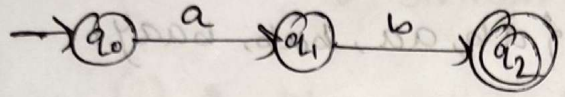
(15) $r_6 = a^*, a^*$

$= a^*$, $L(r_6) = \{\epsilon, a^1, a^2 \ldots a^*\}$

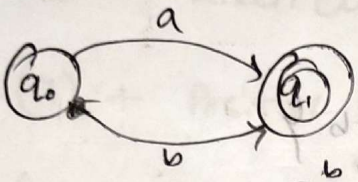
(16) $r_6 = ab^*$ $L(r_6) = \{\epsilon, ab, abab, ababab, \ldots\}$

Regular Expression to

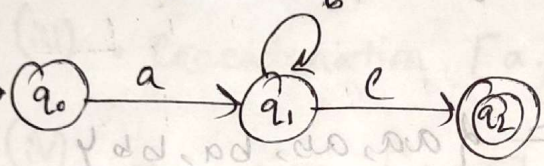
Conversion of Finite Automata (NFA/DFA) to Regular Expression:

(1) 

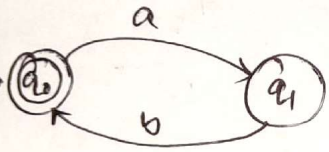
→ a.b

(2) 

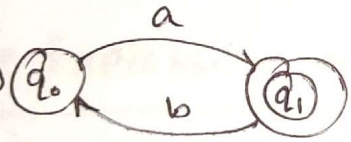
→ a+b

(3) 

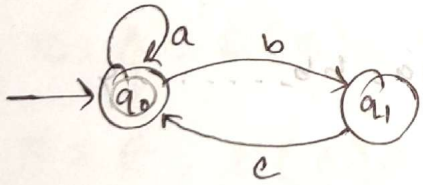
→ ab*c

(4) 

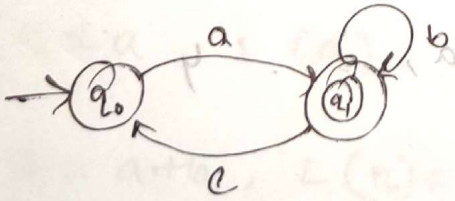
→ (a.b)*

(5) 

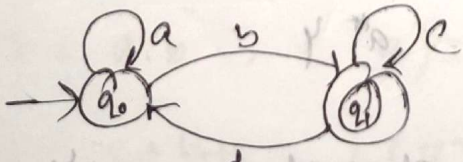
→ a.(ba)*

(6) 

→ (a* + (bc)*)*
= (a + bc)*

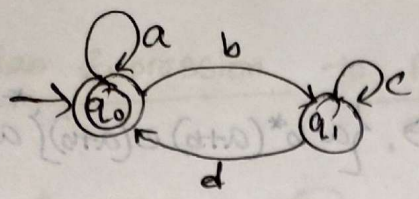
(7) 

→ a(b* + (ca)*)*
= a(b + ca)*

(8) 

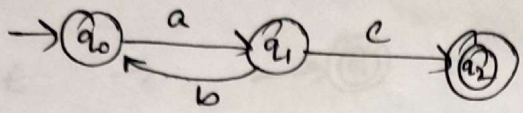
→ a*b(c + da*b)*

(9)



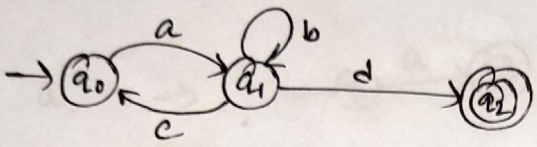
$\rightarrow (a + bc^*d)^*$

(10)



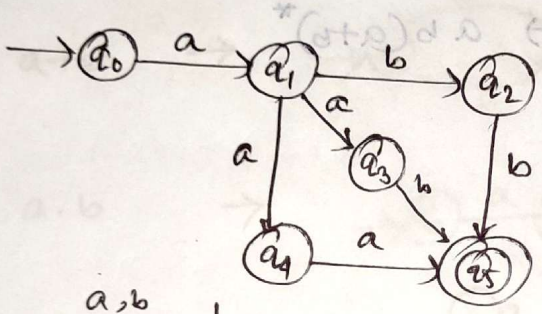
$\rightarrow$ ~~$a(ba+c)^*$~~ $\rightarrow a(ba)^*c$

(11)



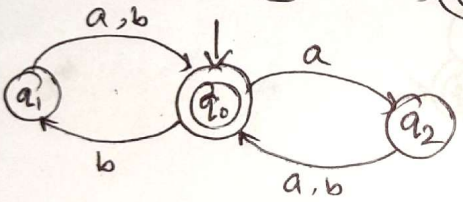
$\rightarrow a(b + ca)^*d$

(12)



$\rightarrow a(bb + ab + aa)$

(13)

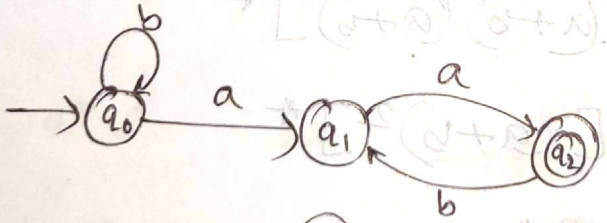


$\rightarrow \{a \cdot (a+b) + b(a+b)\}^*$

$\rightarrow \{(a+b)(a+b)\}^*$

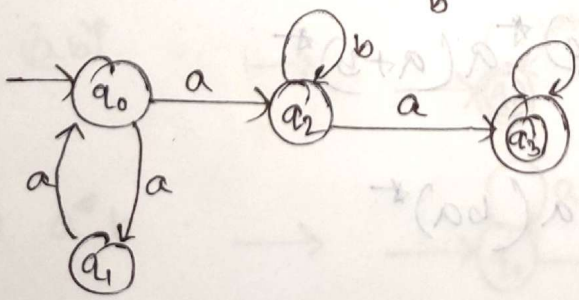
$\rightarrow \{(a+b)^2\}^*$

(14)



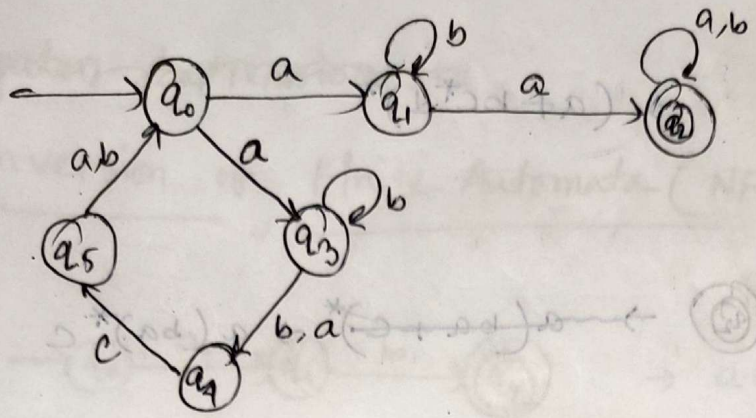
$\rightarrow b^*aa(ba)^*$

(15)



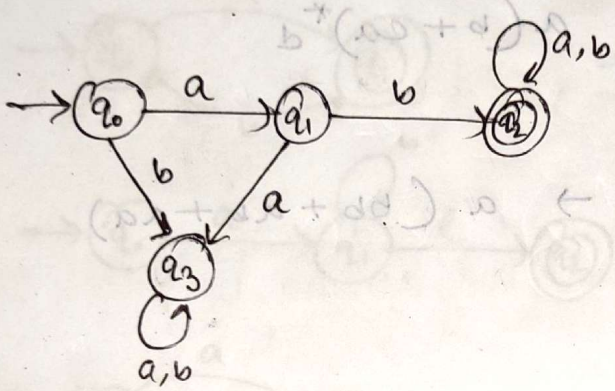
$\rightarrow (aa)^*ab^*a(a+b)^*$

(16)



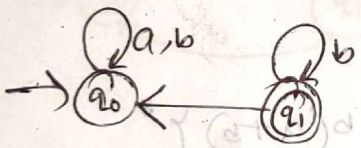
$\rightarrow \{a\,b^*(a+b)\,c(a+b)\}^*\,a\,b^*a(a+b)^*$

(17)



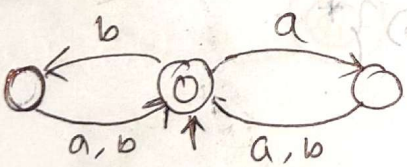
$\rightarrow a\,b(a+b)^*$

(18)



$\rightarrow \phi$

(19)

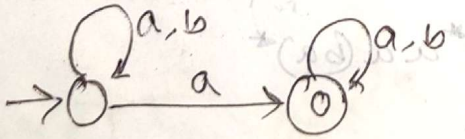


$\rightarrow [b(a+b) + a(a+b)]^*$

$= [(a+b)(a+b)]^*$

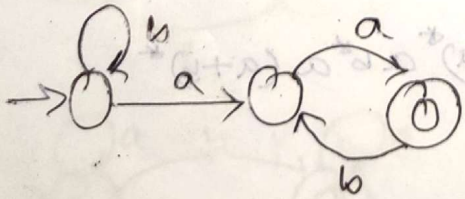
$= [(a+b)^2]^*$

(20)



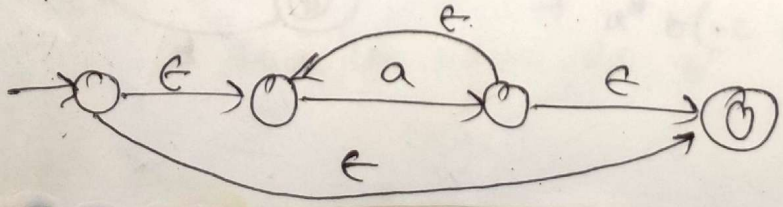
$\rightarrow (a+b)^*a(a+b)^*$

(21)



$\rightarrow b^*aa(ba)^*$

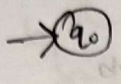
(22)



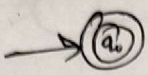
$\rightarrow a^*$

Regular Expression to Finite Automata(NFA / DFA)

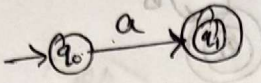
(1) ϕ →



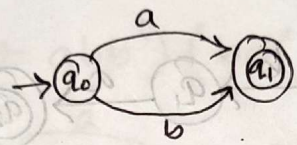
(2) ϵ →



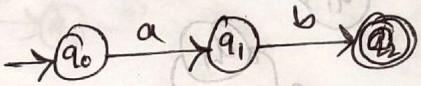
(3) a →



(4) a+b →



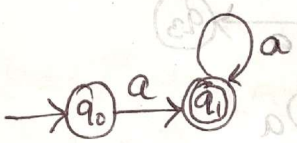
(5) a.b →



(6) a^* →

$L = \{\epsilon, a, aa, aaa, \ldots\}$

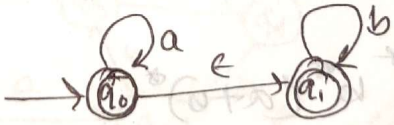
(7) a^+ →



$L = \{a, aa, aaa, \ldots\}$

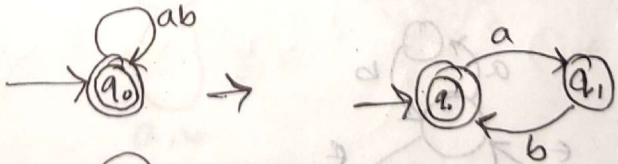
(8) $(a+b)^*$ →

(9) $a^* b^*$ →

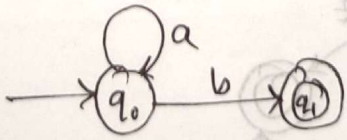


[যেখানে serial maintain হচ্ছে সেখানে পর পর a আসলে accept হবে না]

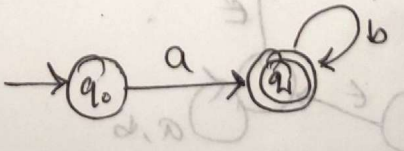
(10) $(ab)^*$ →



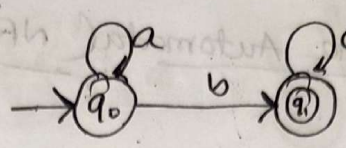
(11) $a^* b$ →



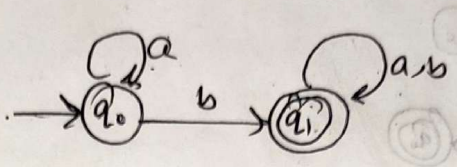
(12) ab^* →



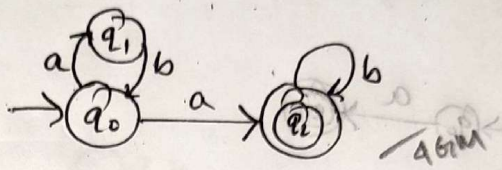
(13) $a^* b e^*$ →



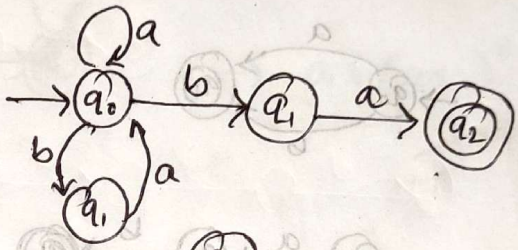
(14) $a^* b (a+b)^*$ →



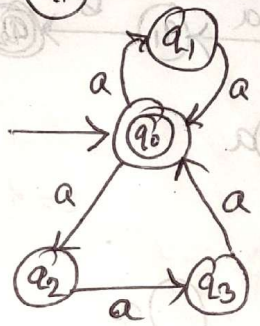
(15) $(ab)^* ab^*$ →



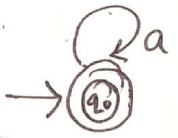
(16) $(a+ba)^* ba$ →



(17) $(aa + aaa)^*$ →

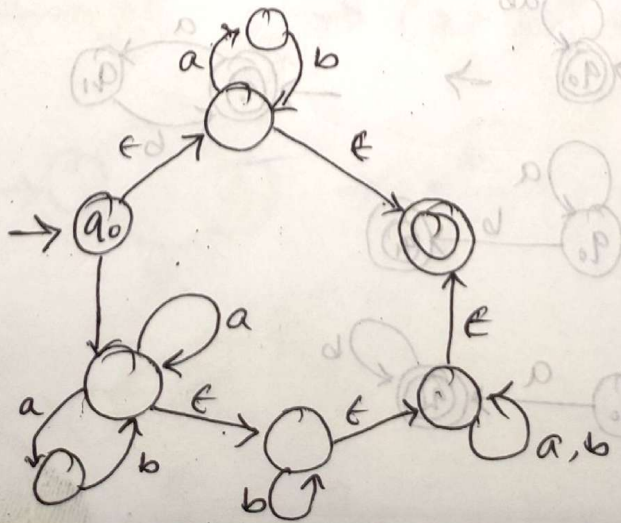


(18) $(a + aaaa)^*$ →

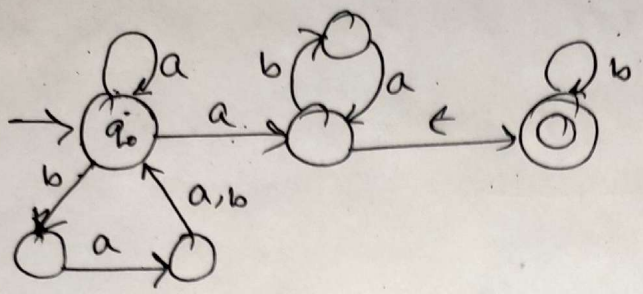


[a^* ই সম্পূর্ণ সকল language কে কভার করবে, তাই $(aaaa)^*$ আর দেখানোর প্রয়োজন নেই।]

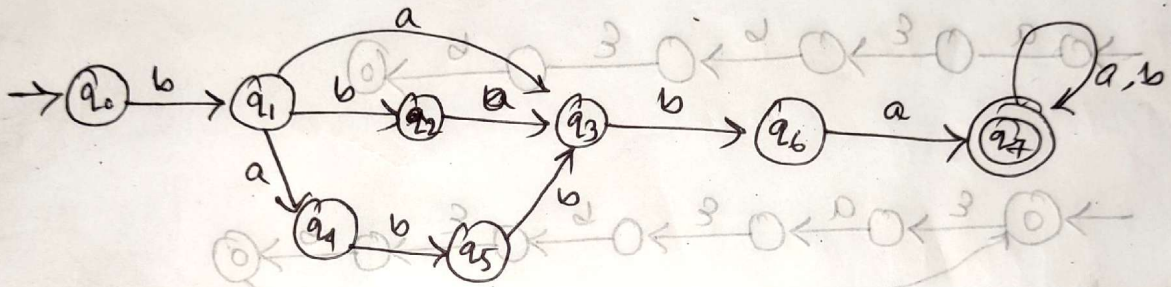
(19) $(ab)^* + (a + ab)^* b^* (a+b)^*$



(20) $[a + ba(a+b)]^* a(ba)^* b^*$

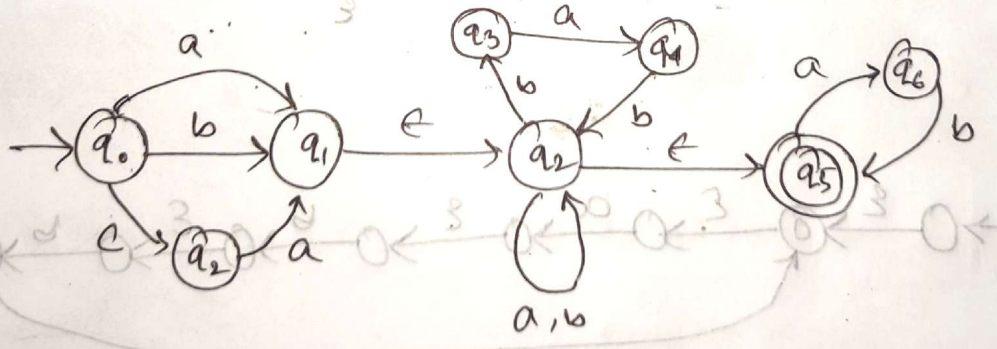


(21) $b(a + ba + abb)(ba(a+b)^*)$



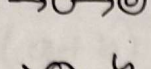
(22) $(a+b+ca)((bab)^* + (a+b)^*)^* (ab)^*$

$= (a+b+ca)(bab + (a+b))^* (ab)^*$

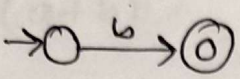


$a(abb)^* \cup b$ to NFA

a :



b :



ab:

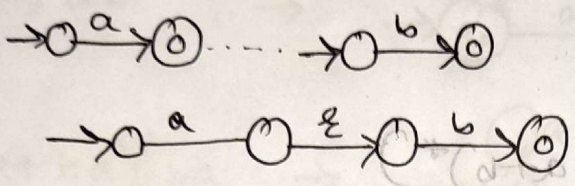
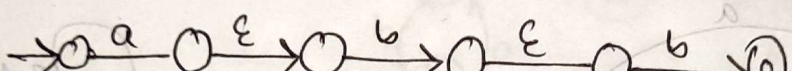
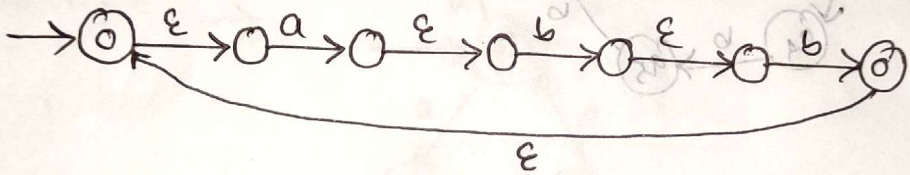


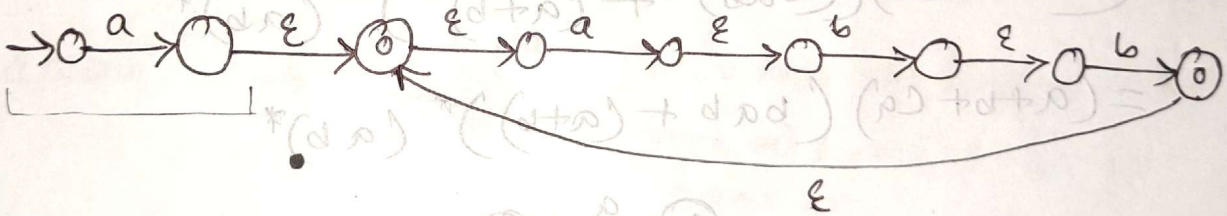
abb:



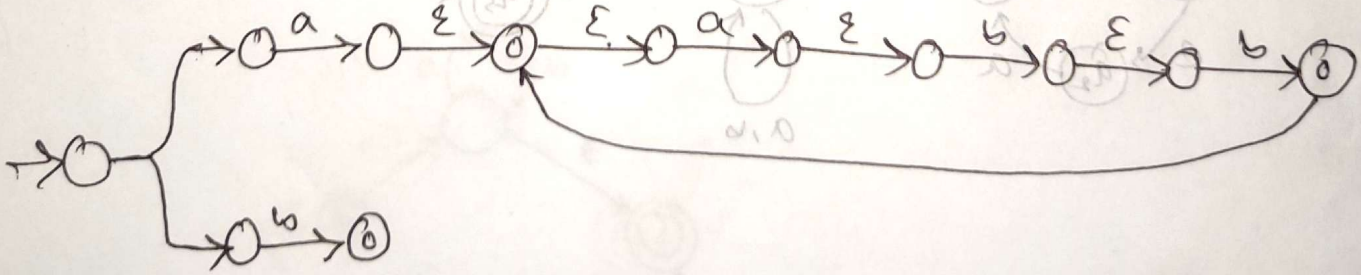
$(abb)^*$



$a(abb)^*$

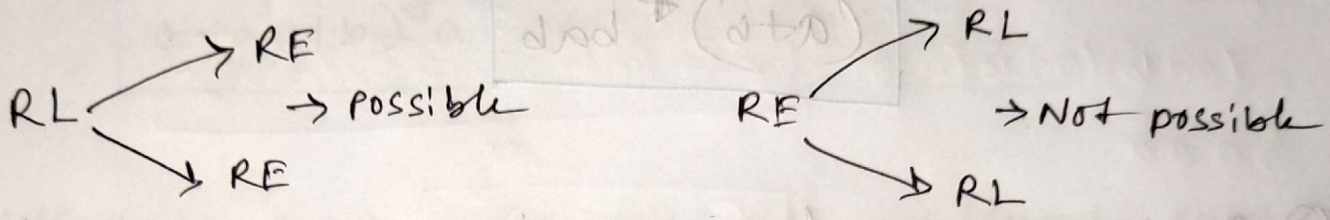


$a(abb)^* \cup b$



Regular Language to Regular Expression

Every regular expression can generate only one regular language but a regular language can be generated by more than one regular expression.



(1) Design a regular expression that represents a language 'L', where L = {a} over the alphabet set $\Sigma = \{a\}$.

RE : $\boxed{a}$

(2) Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$, where every accepted string 'w' starts with substring s = 'abb'

$$\boxed{abb(a+b)^*}$$

(3) Design a regular expression that represent all string over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' ends with substring s = 'bab'?

$$(a+b)^* bab$$

(4) Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$. where every accepted string 'w' contains substring s = 'aba'?

$$(a+b)^* aba (a+b)^*$$

(5) Design a regular expression that represent all strings over ~~the~~ the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with a.

$$a + a(a+b)^* a$$

(6) Design a regular expression that represent all strings over the alphabet $\Sigma = \{a, b\}$ such that every accepted string start and end with same symbol?

$$a + a(a+b)^* a + b + b(a+b)^* b$$

(7) ~~Different~~ start and end with different symbol

$$a(a+b)^* b + b(a+b)^* a$$

(8) every accepted string w, is like $w = xs$. $s =$ 'aaa/bbb' ?

$$(a+b)^* (aaa + bbb)$$

(9) every accepted string w, is like $w = xsx$, $s = aaa/bbb$?

$$(a+b)^* (aaa + bbb) (a+b)^*$$

(10) every string 'w' accepted must be like

(i) $|w| = 3$

$(a+b)^3$

$= (a+b)(a+b)(a+b)$

$\Rightarrow aaa$

$\vdots$

$\Rightarrow bbb$

(ii) $|w| \leq 3$

length 0, len 1, len 2, length 3

$\epsilon + (a+b) + (a+b)^2 + (a+b)^3$

or,

$(a+b+\epsilon)^3$

(iii) $|w| \geq 3$

$(a+b)^3 (a+b)^*$

(11) every accepted string 2nd from the left end is always b,

$(a+b)\ b\ (a+b)^*$

1st, 2nd, 3rd to ∞

(12) 4th from right end is always a

$(a+b)^*\ a\ (a+b)^3$

(5th to ∞) from last, 4th, last 3

(13) Design a regular Expression that represent all string over the alphabet $\Sigma = \{a, b\}$, such that every string 'w' where $|w| = 0 \pmod 3$?

$$[(a+b)^3]^*$$

(14) where $|w| = 3 \pmod 4$

$$(a+b)^3 [(a+b)^4]^*$$

rem 3 | mod 4

(15) where $|w|_a = 0 \pmod 3$

$$b^* + (b^* a b^* a b^* a b^*)^*$$

(16) where $|w|_b = 2 \pmod 3$

$$a^* b a^* b a^* (a^* b a^* b a^* b a^*)^*$$

rem 2 | mod 3